

Notebook

February 8, 2026

1 Атака RSA Blinding

1.1 Введение

RSA (названа в честь своих создателей Ronald Linn Rivest, Adi Shamir and Leonard Adleman) - это пример асимметричной криптосистемы, которая может быть использована для безопасной передачи данных и создания подписей. Хотя от неё постепенно избавляются, её всё ещё можно встретить во многих продуктах и системах, поэтому есть смысл понимать, как она работает. Начнем с основ

1.2 Группы и Поля

Группа - это множество $\mathbb{G}$ определенной на нем операцией с двумя аргументами $*$, $\forall a, b \in \mathbb{G} \Rightarrow a * b \in \mathbb{G}$ и свойствами:

- Ассоциативности ($\forall a, b, c \in \mathbb{G}, (a * b) * c = a * (b * c)$)
- Наличия единичного элемента ($\exists e \in \mathbb{G} : \forall a \in \mathbb{G}, a * e = e * a = a$)
- Наличия обратного элемента ($\forall a \in \mathbb{G}, \exists b \in \mathbb{G} : a * b = e = b * a$)

Группа называется коммутативной (абелевой), если она удовлетворяет свойству коммутативности:

- $\forall a, b \in \mathbb{G}, a * b = b * a$

Т.е., если у нас есть некоторое множество, результат применения оператора к любым двум элементам также содержится в этом множестве, порядок вычисления выражений не важен, есть некий элемент, который не изменяет другие элементы (1) и для любого элемента всегда есть такой парный, что при применении к ним оператора получается единичный (1), то у нас есть группа. Если ещё и можно поменять местами аргументы при вызове оператора и ничего не изменится, то аж абелева.

Если мы используем аддитивную нотацию (ставим плюс), то группа называется аддитивной. Если мультипликативную (ставим знак умножения) - мультипликативной.

Давайте рассмотрим простой пример аддитивной группы целых чисел по модулю некоторого числа n . Например, если $n = 5$, то группа содержит элементы $\{0, 1, 2, 3, 4\}$. Если происходит переполнение (результат меньше 0 или больше или равен n), мы тут же добавляем или вычитаем $k * n, k \in \mathbb{N}$ из результата, чтобы он

снова был в множестве. Как видно, 0 - единичный элемент, 1 - обратный элемент к 4, а 2 - к 3. Ассоциативность очевидна, а поскольку мы легко можем менять элементы местами при сложении, то эта группа ещё и абелева.

О полях можно думать как о группах с двумя операциями (это не совсем верно, но так проще вникнуть). Допустим у Вас есть множество с двумя операциями $(+, *)$. Оно будет полем, если:

1. Для обеих операций оно является абелевой группой, за исключением единичного элемента аддитивной группы, который не входит в мультипликативную (у 0 же не может быть обратного элемента).
2. Действует закон дистрибутивности: $a * (b + c) = a * b + a * c$.

В качестве примера давайте посмотрим на поле F_n , где $n = 5$:

1. Сложение осталось таким же, как и в прошлом примере
2. Таблица умножения:

	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

Можно видеть, что в каждом ненулевом ряду есть по 1, так что у каждого элемента есть обратный. Если удалить ряд и столбец, содержащие нулевой элемент, то как раз получится таблица мультипликативной группы по модулю n или $\mathbb{Z}_n^*$, которая содержит элементы $\{1, 2, 3, 4\}$. Здесь же можно заметить интересную особенность групп. Если порядок (количество элементов) в группе не простое, то можно генерировать подгруппы. Это группы, которые используют ту же операцию, что и оригинальная (например, умножение по модулю 5), но состоят из подмножества элементов оригинальной группы. В данном случае элементы $\{1, 4\}$ образуют такую подгруппу, т.к. $4 \cdot 4 = 1 \mod 5$ (получаем замкнутое по умножению множество). Подгруппы упрощают вычисление дискретного логарифма (но об это позже). Из-за этого в криптографии часто используются большие безопасные простые числа $p = 2 * q + 1$, где q - это другое простое число. Таким образом получается всего 2 нетривиальных подгруппы с порядками 2 и q .

1.3 RSA

RSA использует мультипликативную группу по модулю $N = pq$, где p и q - простые числа. Степень мультипликативной группы (по-сути, её мощность) может быть вычислена при помощи функции Эйлера для составного числа из двух простых: $\varphi(N) = (p - 1)(q - 1)$. Функция считает количество натуральных чисел меньше N , которые не кратны p или q .

Например, если мы возьмем $N = 13 \cdot 17 = 221$, то элемент 26 не состоит в мультипликативной группе, т.к. $26 \cdot 17 = 0 \bmod 221$, а 0 не в группе. Т.е. надо исключить все элементы кратные p (таких q) и q (таких p). Т.е. всего элементов в группе будет $p \cdot q - p - q + 1 = (p - 1) \cdot (q - 1)$ (добавляем 1, потому что 0 посчитали до этого два раза).

Поскольку все остальные числа взаимно просты с N , они состоят в мультипликативной группе. Как мы знаем, если возвести любой элемент конечной мультипликативной группы в степень этой группы, то получим нейтральный элемент (единицу): $a^{\varphi(N)} = 1, a \in Z_N^*$. Поэтому в RSA используют два числа e (открытая экспонента) и d (закрытая экспонента), такие что $ed = 1 \bmod \varphi(N)$. Пара чисел (e, N) используется как открытый ключ, а (d, N) как закрытый. Вычисление d из открытого ключа является сверхполиномиальной задачей (NP), если не были сгенерированы слабые N , d или e . Одним из способов решения является факторизация N в произведение p и q .

Пусть дан открытый текст (число) $M, M < N$, открытый ключ (e, N) и закрытый ключ (d, N) , шифрование и расшифрование осуществляются следующим образом:

Шифрование $C = M^e \bmod N$

Расшифрование

$M = C^d \bmod N$

Проверка корректности:

$C^d \bmod N = M^{ed} \bmod N = M^{ed \bmod \varphi(N)} \bmod N = M^1 \bmod N = M \bmod N$

1.4 Подготовка

Попробуем немного поработать с RSA. Если ещё не установили, установите Pycryptodome. На Linux и Windows должна сработать следующая команда (Предварительно надо установить python 3 и pip, но я надеюсь, что вы справились с этим самостоятельно):

```
[ ]: !python3 -m pip install --user pycryptodome
```

После установки надо перезапустить ядро jupyter (круговая стрелка рядом с "Run"). Если возникнут проблемы, загляните в документацию: [Pycryptodome installation](#).

1.5 Примитивная реализация RSA

Давайте сделаем простейшую версию RSA. Будем использовать открытую экспоненту $e = 65537$. Обычно используют эту константу, потому что она переполняет модуль даже при открытом тексте $M = 2$ и у нее удобное двоичное представление $65537_{10} = 10000000000000001_2$, которое позволяет эффективно возводить число в степень, используя алгоритм "Square and multiply".

Сначала сгенерируем p и q . Функция `getStrongPrime` дает возможность выбрать количество бит в генерируемом простом числе и проверяет, что $(p-1, e) = 1$

```
[ ]: try:
      from Crypto.Util.number import getStrongPrime, inverse, bytes_to_long, long_to_bytes
    except ImportError:
      print ("Pycryptodome not installed")
```

```
[ ]: e=65537
     p=getStrongPrime(1024,e=e)
     q=getStrongPrime(1024,e=e)
```

```
[ ]: N=p*q
     phi=(p-1)*(q-1)
     d=inverse(e,phi)
     public_key=(e,N)
     private_key=(d,N)
```

Мы успешно сгенерировали ключи, теперь давайте зашифруем сообщение, расшифруем закрытый текст и проверим, что получили то же самое

```
[ ]: M=bytes_to_long(b'Hello, RSA!')
     C=pow(M,e,N)
     print ('C:',hex(C))
     M1=pow(C,d,N)
     assert M1==M
     print ('M1:',long_to_bytes(M1))
```

Создание подписи - обратная операция к шифрованию.

$$\text{Sign}(M) \equiv \text{Dec}(M), \text{Check}(S) \equiv \text{Enc}(S)$$

Таким образом любой, владеющий открытым ключом, может проверить правильность подписи, а создать её может только сторона, у которой есть закрытый ключ. Поздравляю, теперь вы знаете, как шифровать и создавать подписи при помощи RSA. Далее рассмотрим одно из его интересных свойств.

1.6 RSA Blinding

RSA - это гомоморфное шифрование по отношению к операции умножения. Отображение является гомоморфизмом групп, если оно сохраняет отношения между элементами. Если ничего не понятно, не беспокойтесь, я в первый раз, когда услышал, тоже ничего не понял. Что это значит на практике: пусть у вас есть два элемента группы G_1 (x, y) и вы применяете к ним гомоморфное отображение, они будут также связаны в новой группе G_2 (для RSA $G_1 = G_2$):

$$\varphi(x \cdot y) = \varphi(x) \times \varphi(y)$$

Для шифрования RSA:

$$Enc(M_1 \cdot M_2) = Enc(M_1) \times Enc(M_2)$$

То же самое верно и для расшифрования:

$$Dec(C_1 \times C_2) = Dec(C_1) \cdot Dec(C_2)$$

Протестируем это свойство в python

```
[ ]: class BasicRSA:
    def __init__(self, e,p,q):
        self.e=e
        self.p=p
        self.q=q
        self.N=p*q
        self.d=inverse(e, (p-1)*(q-1))

    def encryptNumber(self, m):
        return pow(m,self.e,self.N)

    def decryptNumber(self, c):
        return pow(c,self.d,self.N)

brsa=BasicRSA(e,p,q) #we created these parameters earlier
m1=2
m2=3
m3=m1*m2
c1=brsa.encryptNumber(m1)
c2=brsa.encryptNumber(m2)
print ('c1:',c1)
print ('c2:',c2)
c3=(c1*c2)%brsa.N
print('c3:',c3)
m3_dec=brsa.decryptNumber(c3)
print ('m3: %d, m3_dec: %d'%(m3,m3_dec))
assert m3_dec==m3
```

1.7 Атакуем сервер

Теперь попробуйте применить эти знания к уязвимому серверу. Вы можете приконнегтиться, используя nc cryptotraining.zone 1337 или при помощи питоновских сокетов.

```
[ ]: import socket
import re
class VulnServerClient:
    def __init__(self, show=True):
        """Инициализация, подключаемся к серверу"""
```

```

self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
self.s.connect(('cryptotraining.zone',1337))
if show:
    print (self.recv_until().decode())
def recv_until(self,symb=b'\n>'):
    """Получение сообщения с сервера, по умолчанию до приглашения_
↪К вводу команды"""
    data=b''
    while True:

        data+=self.s.recv(1)
        if data[-len(symb):]==symb:
            break
    return data
def get_public_key(self,show=True):
    """Получение открытого ключа с сервера"""
    self.s.sendall('public\n'.encode())
    response=self.recv_until().decode()
    if show:
        print (response)
    e=int(re.search(r'(?<=e: )\d+',response).group(0))
    N=int(re.search(r'(?<=N: )\d+',response).group(0))
    self.num_len=len(long_to_bytes(N))
    return (e,N)

def signBytes(self,m,show=True):
    """Получение подписи для выбранного сообщения в байтах с_
↪сервера"""
    try:
        num_len=self.num_len
    except AttributeError:
        print ('You need to get the public key from the server_
↪first')
        return
    if len(m)>num_len:
        print ("The message is too long")
        return
    if len(m)<num_len:
        m=bytes((num_len-len(m))*[0x0]) +m
    hex_m=m.hex().encode()
    self.s.sendall(b'sign '+hex_m+b'\n')
    response=self.recv_until().decode()
    if show:
        print (response)
    if response.find('flag')!=-1:
        print('You tried to submit \'flag\'')
        return None

```

```

        signature_hex=re.search(r'(?<=Signature: )[0-9a-
f]++',response).group(0)
        signature_bytes=bytes.fromhex(signature_hex)
        return bytes_to_long(signature_bytes)

    def signNumber(self,m,show=True):
        """Получение подписи с сервера для выбранного сообщения в
        ↳числовом представлении"""
        try:
            num_len=self.num_len
        except AttributeError:
            print ('You need to get the public key from the server,
            ↳first')
            return
        return self.signBytes(long_to_bytes(m,num_len),show)

    def checkSignatureNumber(self,c,show=True):
        """Проверка сигнатуры (на сервере) для подписи в числовом
        ↳представлении"""
        try:
            num_len=self.num_len
        except AttributeError:
            print ('You need to get the public key from the server,
            ↳first')
            return
        signature_bytes=long_to_bytes(c,num_len)
        self.checkSignatureBytes(signature_bytes,show)

    def checkSignatureBytes(self,c,show=True):
        """Проверка сигнатуры (на сервере) для подписи в байтовом
        ↳представлении"""
        try:
            num_len=self.num_len
        except AttributeError:
            print ('You need to get the public key from the server,
            ↳first')
            return
        if len(c)>num_len:
            print ("The message is too long")
            return

        hex_c=c.hex().encode()
        self.s.sendall(b'flag '+hex_c+b'\n',)
        response=self.recv_until(b'\n').decode()

```

```

    if show:
        print (response)

    if response.find('Wrong')!=-1:
        print('Wrong signature')
        x=self.recv_until()
        if show:
            print (x)
        return
    flag=re.search(r'CRYPTOTRAINING\{.*\}',response).group(0)
    print ('FLAG: ',flag)

def __del__(self):
    self.s.close()

```

```

[ ]: vs=VulnServerClient()
      (e,N)=vs.get_public_key()

```

Вы можете подписывать сообщения при помощи методов `signNumber` (подписать число) и `signBytes` (подписать сообщение из байтов)

Проверять подпись можете при помощи методов `checkSignatureNumber` и `checkSignatureBytes`.

Ваша цель - получить правильную подпись для сообщения 'flag'.

Помните, что RSA - это гомоморфизм и решите задание.

Удачи!

```

[ ]: vs.signBytes(b'flag')

```

```

[ ]:

```